DEEP AUDIT AND UPGRADE REPORT

IC Intelligence Center

A full engineering, security, and UI/UX review of the Standard-autonomous repository: what is genuinely built, what is broken, what to change, what to add, and how to run it like big-company software.

Standard-autonomous Repository Review

Engineering - Security - Pipeline - UI/UX - Market Expansion

OCTOBER 2026

Table of Contents

1. Executive Summary and Audit Verdict 1

2. Repository Audit: What IC Is Today 3

3. What Makes Big-Company Software: The Engineering Review 7

4. Reorganization: A Big-Company File Structure 10

5. The Unified Data Pipeline: From Scraping to CRM 13

6. UI/UX Blueprint: From Dashboard to Agency OS 17

7. Paperclip, REA, and pstack: What They Are and How Your Team Uses Them 20

8. Market Expansion: High-Pain Categories Beyond the Three Verticals 23

9. Four-Week Engineering Plan and KPIs 25

1. Executive Summary and Audit Verdict

This report is a full engineering review of the **Standard-autonomous** repository (github.com/shaikhsarib/Standard-autonomous), the codebase behind IC, the Intelligence Center: an autonomous lead-generation and sales-development platform written in Python and FastAPI. The review covers the questions you asked directly: is the infrastructure real, is it genuinely all-in-one, what makes software feel like big-company software, is the UI and feature set clean, is the security acceptable, and what exactly should be changed and added. Nothing in your repository was modified; every finding below is backed by evidence captured from the code, the test suite, and the documentation during this audit. The review also folds in research from more than twenty-five external sources: nineteen UI and component libraries and design galleries, eight engineering repositories (Scrapling, Firecrawl, google-maps-scraper, treg, Chatwoot, rakazo, Twenty, harness2), the Slack engineering blog, and the pstack, Matt Pocock, superpowers, and ECC skill systems that are vendored inside your own repo.

51	33 / 51	0	6,528
agent skills vendored in .agents/skills	tests passing, not the 51 the README claims	CI workflows, Dockerfiles, or LICENSE files	lines in the single dashboard.html file

The verdict in one paragraph: **IC is a real system with unusually good bones, and it is not yet industry-grade.** The architecture is genuinely well designed for a solo build: a five-agent org chart (Scout, Auditor, Copywriter, ApprovalGate, Closer) with typed contracts, a pstack-style evidence verifier that refuses unproven claims, an ECC security shield, a human approval gate before any outbound message, and a token-driven dashboard with light and dark themes. Those are the parts that big companies get right, and you already have them. What breaks the industry-grade claim is the verification and operations layer: the documented storage module was never committed, so five of ten test files cannot even be imported; one test fails and eight error out; there is no CI, no Docker, no LICENSE, no authentication on the dashboard, wildcard CORS, ninety runtime data and log files committed to git, and three parallel scraper trees left over from earlier versions. In other words: the product architecture is ahead of typical solo work, while the engineering discipline behind it is behind big-company work. Closing that gap is a four-week job, and Chapter 9 gives the exact plan.

The five most important findings

1. **The test suite is broken while the README claims it is green.** The README advertises a 51-test, all-mocked, zero-cost suite. Executing pytest today yields 33 passed, 1 failed, 8 errors, and 5 files that fail at import because they import src/storage/repository.py, a module documented in folder_structure.md but never committed. This is the single most damaging finding, because your whole pitch to clients is that IC proves its claims with evidence; the repo currently fails its own evidence standard.

2. **The architecture has drifted from the documentation.** folder_structure.md describes a v2.0 tree with src/storage at its center. The actual src/ directory has no storage package, and main.py still imports the older v1 modules at the repository root (core/, services/, scrapers/, delivery/). Two generations of the same system live side by side, and a third scraper set exists in outreach/. Nothing tells a new engineer which tree is alive.

3. **Security is developer-grade, not production-grade.** The FastAPI dashboard has zero authentication endpoints, allow_origins set to the wildcard "*", and uvicorn reload=True in the production entry point. Every lead contains personal data stored as plain JSON files. For a local demo this is fine; for an agency handling client data it is not shippable.

4. **The integrations exist but run in degraded mode.** Scrapling, Firecrawl, and the gosom google-maps-scraper are all wired into src/scrapers/ with sensible fallbacks, but none of those libraries appear in requirements.txt, so out of the box the system silently runs its fallback paths instead of the real tools, and no passing test covers any of them.

5. **The UI is closer to big-company quality than expected, and there is a precise path to finish it.** The dashboard already uses CSS custom properties for light and dark themes, monospace/sans token pairs, and react-bits-style SpotlightCard patterns. Chapter 6 gives a ranked, source-backed list of twelve upgrades, plus ten anti-slop guardrails, that turn it into a Linear or Vercel-grade console without a rewrite.

What to change and what to add, at a glance

Chapter 4 gives the full reorganization plan, but the short answer to "what should I change" is: restore the missing storage layer, merge the v1 root modules into src/, delete the duplicate outreach/ scrapers, untrack data/ and logs/ from git, add a LICENSE and a CI workflow that runs pytest plus the ECC AgentShield scan, and put one token gate in front of the dashboard. The short answer to "what should I add" is: an idempotent job queue so scraping and sending survive restarts (the Slack engineering lesson), Chatwoot as the two-way inbox that turns fire-and-forget outreach into real conversations, Twenty as the CRM data model with its table and kanban UI patterns, the treg gateway pattern so vendor keys never live in agent code, and the twelve UI upgrades from Chapter 6. Chapters 5 through 8 give the pipeline design, the UI blueprint, plain-language explanations of Paperclip, REA and pstack, and nine new market verticals with quantified pain.

> *Bottom line: you do not need to rebuild IC. You need to make it tell the truth (fix tests and docs), lock the doors (auth and secrets), finish the merge (one module tree), and then invest the freed energy in the inbox, the CRM, and the UI polish that will make it feel like the big-company software you are comparing it to.*

2. Repository Audit: What IC Is Today

2.1 System inventory

IC (Intelligence Center) v2.5 is a self-described "production-grade, multi-agent Intelligence Center, Technical Audit, and Autonomous SDR platform" inspired by Paperclip-style workforce management and REA-style reverse engineering. The repository is 109 MB, contains roughly 70,000 lines of Python across 304 files, one 6,528-line dashboard HTML file, and a 51-skill agent stack vendored under .agents/skills (3,592 markdown files plus 1,017 JavaScript files). The whole history is a single commit, which means there is no review trail, no tagging, and no way to bisect regressions. The following inventory maps what each part actually does, based on reading the code rather than the documentation.

Layer	Location	What it actually does	State
Domain models	src/models/lead.py, conversation.py	Pydantic v2 Lead, BusinessProfile, AuditReport, OutreachDraft, LeadStatus; timezone-aware UTC	Healthy
Agent contracts	src/core/contracts.py	HandoffEnvelope, AgentRole, ContractViolationError: typed hand-offs between agents	Healthy
Pipelines	src/pipelines/	discovery (find, sanitize, dedupe, store), outreach (enrich, dispatch), chat (multi-turn SDR)	Healthy
Scrapers (v2)	src/scrapers/	base + gmaps + web_discovery + firecrawl + scrapling + gosom wrappers	Wired, deps missing
Services	src/services/	workforce, analyzer, site_inspector, visual_auditor, evidence_verifier, inbound_closer, vertical_playbooks, agency_swarm, paperclip_bridge, treg and firecrawl clients, harness engine, cadence	Mostly healthy
ECC shield	src/agents/ecc_shield.py	credential, injection, and dependency gates plus outbound pitch compliance	Healthy, unenforced in CI
API + UI	src/api/server.py, templates/dashboard.html	FastAPI app (932 lines) + 4-step command center (6,528 lines) with Paperclip drawer	Healthy, unauthenticated
Storage	src/storage/repository.py	LeadRepository and SessionRepository: documented in folder_structure.md, imported by 5 test files	**Missing, never committed**
v1 root modules	core/, services/, scrapers/, delivery/, analytics/	engine.py, llm_client.py, scrapper_module.py, delivery_module.py, analytics.py; still imported by main.py and cli.py	Alive, duplicated

Layer	Location	What it actually does	State
Third scraper set	outreach/	gmaps_scraper, sulekha_scraper, email_finder, pain_point_analyzer, reply_tracker	Orphaned duplicate

Table 1. IC module inventory as found in the repository (evidence-based).

The five-agent org chart is the heart of the product and it is genuinely implemented, not just described. ScoutAgent discovers businesses through Google Places, OSM Overpass, and CSV sources with deduplication and name sanitization. AuditorAgent runs the REA-inspired site inspection described fully in Chapter 7. CopywriterAgent turns verified audit findings into Hinglish or English pitches using per-vertical playbooks and the pitch_claims.yaml claim registry. ApprovalGate is a real human-in-the-loop staging queue in the dashboard, which is the correct design for cold outreach both ethically and legally. CloserAgent (branded Hermes in the UI) classifies inbound replies into intents such as pricing objection, competitor objection, or demo request, and drives prospects toward a booking. Each hand-off is typed through contracts.py, and the evidence_verifier wraps audit results into PstackAuditBundle objects whose claims carry a proof_reference and a pstack_status of PROVEN_TRUE. This is the part of IC that already thinks like big-company software.

2.2 What is genuinely strong

- **Typed agent contracts.** HandoffEnvelope with AgentRole and a ContractViolationError means agents cannot pass arbitrary junk to each other. Most solo projects never do this; it is the foundation that lets you add agents without chaos.
- **Evidence verification before persuasion.** evidence_verifier.py builds proof bundles where every claim (TTFB in milliseconds, SSL status, missing WhatsApp widget) carries evidence_type, proof_reference, and an optional reproducibility_command. This is your anti-hallucination engine and it directly implements the pstack "prove it works" principle.
- **Human approval by default.** Outreach is staged, previewed side-by-side for WhatsApp and email, and only sent after a human one-click approval, with a dry-run mode in the CLI. This is both the ethically correct pattern and the pattern that survives platform policy (WhatsApp Business messaging policy, spam filters).
- **Multi-provider LLM abstraction with retry.** BaseLLMClient with Gemini, Anthropic, and Ollama implementations, automatic assistant-to-model role mapping for Gemini, and exponential backoff on 429 and 5xx. Adding a provider is one subclass and one branch.
- **None-safe and injection-aware delivery.** Every delivery field is coerced through a _s() helper, HTML escaping is applied to Telegram messages, and the ECC shield module encodes credential, SQL-injection, eval, and dependency guards as reviewable code.
- **A real design token system in the dashboard.** Light and dark themes defined through CSS custom properties (--bg-app, --bg-card, --bg-sidebar, font-sans, font-mono), which is exactly the token architecture Twenty uses to make its UI feel native.

- **Vertical playbooks as data.** vertical_playbooks.py with detect_vertical() plus pitch_claims.yaml means adding a new market (Chapter 8) is a data change, not a code change.
- **The skills stack is real and vendored.** All 51 skills including pstack, superpowers, mattpocock-skills, ecc, treg, harness2, ruflo, and wayfinder are present under .agents/skills/, and several (grilling, to-spec, verification-before-completion) are referenced in code docstrings as the standards the code was written against.

2.3 Critical defects, with evidence

The defects below are ordered by how much they damage trust in the system. Each one was reproduced during this audit, and each maps to a fix in Chapter 9. The pattern behind all of them is the same: the architecture was designed with the right principles, but the last mile of discipline (commit the storage file, run the tests before writing the README, delete the old tree) did not happen. That is precisely the gap the vendored superpowers skill "verification-before-completion" exists to close: no completion claim without fresh verification evidence.

#	Defect	Evidence	Severity
D 1	Test suite broken at import; README overclaims	pytest: 33 passed, 1 failed, 8 errors; 5 files abort with ModuleNotFoundError: src.storage.repository. README claims a green 51-test suite.	P0 - trust
D 2	Documented storage layer missing	folder_structure.md documents src/storage/repository.py (LeadRepository, SessionRepository); the file is absent from the repository.	P0 - architecture
D 3	Dual module trees (v1 root + v2 src/)	main.py imports core.engine, services.llm_client, scrapers.scrapper_module, delivery.delivery_module; other modules import src.*. Both trees alive, no deprecation notes.	P1 - maintenance
D 4	Runtime data and logs committed to git	89 files under data/ and logs/ tracked in git (9.8 MB + 572 KB), while folder_structure.md claims "data/ ignored by git". Leads with PII are in version history.	P0 - security
D 5	No authentication on dashboard/API	Zero matches for Depends/api_key/Bearer in server.py; CORS allow_origins=["*"]; uvicorn reload=True in run_server.py; /docs exposed.	P0 - security
D 6	No CI, no container, no LICENSE	No .github/workflows, no Dockerfile, no docker-compose, no LICENSE file. Nothing prevents regression or defines reuse terms.	P1 - operations

#	Defect	Evidence	Severity
D 7	Scraping integrations run degraded by default	scrapling and firecrawl absent from requirements.txt; scrapling_scraper.py silently falls back when the import fails; zero passing tests cover src/scrapers/.	P1 - capability
D 8	Docs and metadata slop	README links point to a local Windows path (file:///c:/Users/JUBER/...); single-commit history; versioned as v2.5 in README but folder_structure.md says v2.0.	P2 - polish

Table 2. Critical defects found during the audit (all reproducible).

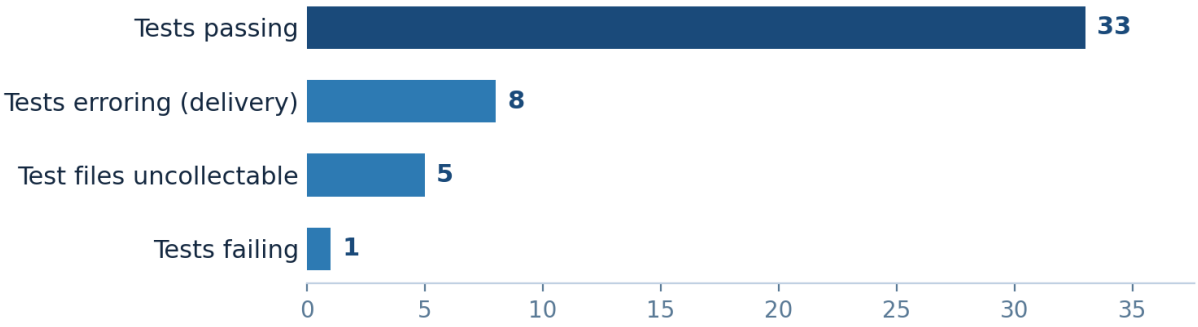


Figure 1. README claim versus measured reality of the test suite (pytest run during this audit).

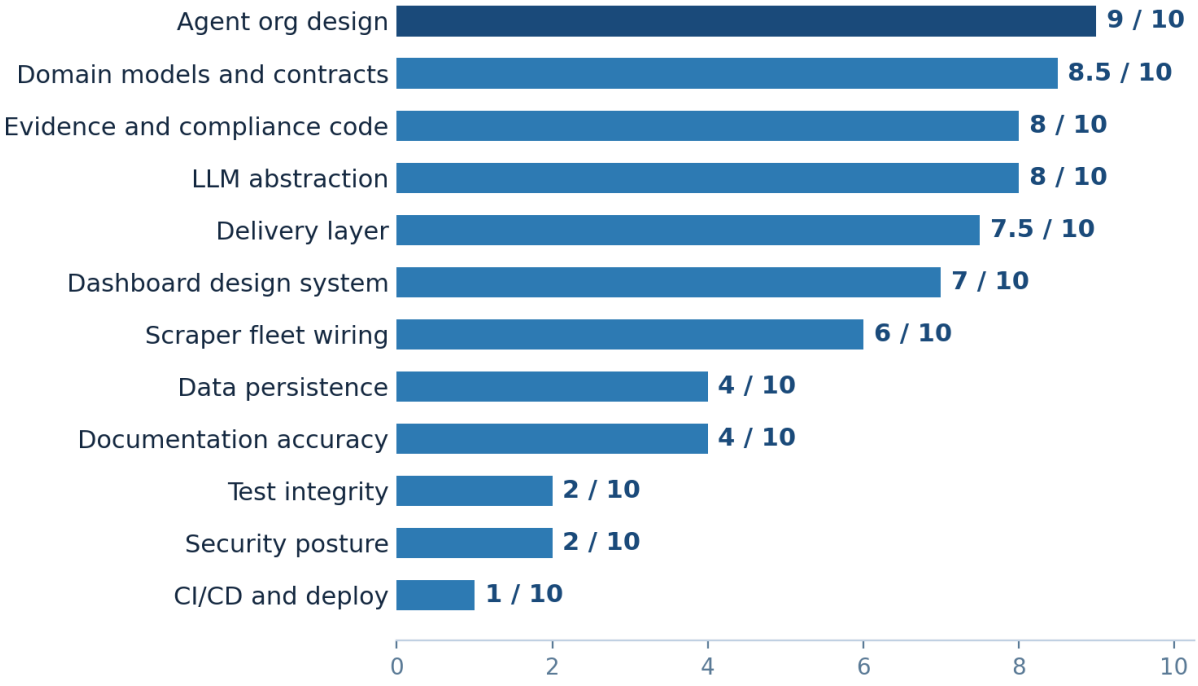


Figure 2. Audit scorecard across eleven engineering dimensions (0-10, reviewer scoring with evidence).

2.4 Is it really all-in-one? An honest answer

The honest answer has three parts. First, as a **lead-generation and SDR operating system**, IC genuinely is all-in-one: discovery, audit, proof, copywriting, human approval, multi-channel dispatch, inbound reply handling, and analytics all exist in one codebase with typed hand-offs, which is more than most agencies ever glue together. Second, as a **general business operating system** it is not all-in-one yet, and should not try to be: it has no invoicing, no client-facing portal, no project management, no knowledge base, and it should integrate those (Chatwoot, Twenty, or SaaS) rather than rebuild them. Third, the "all-in-one" claim currently rests on code paths that have never executed end-to-end under test, because the storage layer the pipelines depend on is the missing module. An all-in-one system that cannot prove its own persistence layer works is, from a client trust perspective, a demo. Fixing D1 and D2 is therefore not housekeeping; it is what converts the demo into the product you can sell.

What makes software feel like "big-company software" is not the number of features. It is the presence of boring, invisible properties: every change is verified by a machine before merge, every state change is idempotent and observable, every tenant is partitioned, every secret is injected rather than typed, every deploy is repeatable from a container, and the documentation matches reality because CI fails when it drifts. IC has the vocabulary of these properties (contracts, proof bundles, shields) but not yet the enforcement (CI gates, queues, containers). Chapter 3 turns each invisible property into a concrete IC work item, and that conversion, not any new feature, is what will make the system feel the way Twenty or Linear feels.

3. What Makes Big-Company Software: The Engineering Review

This chapter answers your question "what really makes this genuine working and industry level and big-company software like" by naming the invisible properties that large engineering organizations build, and scoring IC against each. The properties come from three sources: the Slack engineering blog (the most documented real-time messaging architecture in the industry), the practices encoded in the superpowers and ECC skills you already vendored, and standard multi-tenant SaaS practice. The good news is that none of them require a rewrite of IC; they are additions and gates, not redesigns.

3.1 Eight Slack-scale principles, applied at IC scale

Principle (what Slack does)	IC application (what you build)
Durable job queue as the backbone: every side-effectful operation is a retry-safe background job with idempotent handlers.	Move scraping, auditing, copywriting, and sending into queued jobs (arq or Celery + Redis). A crashed run resumes; the API never blocks on Playwright.
At-least-once delivery with idempotent consumers: Events API retries and tags every callback with an event_id for dedupe.	Every WhatsApp/email send carries a dedupe key (lead_id + template + time window); consumers store seen event IDs; webhook receivers return 200 fast, then process.
One uniform event model, typed and versioned; some events deliberately ephemeral.	Define lead.created, audit.completed, draft.approved, outreach.sent, reply.received as versioned internal events; typing/heartbeat signals stay in memory only.
Edge gateway separated from core: stateful connections terminate at the edge; core processing can restart freely.	Terminate Twilio/Chatwoot webhooks in a thin edge route that authenticates and enqueues only; audits and LLM calls live in restartable workers.
Tiered rate limiting per method per tenant, with 429 + backoff as a first-class response.	Token buckets per tenant and per provider: Google Places quota, WhatsApp Business tier, Gemini RPM. Log every limit hit as an event you can show clients.
Cache at the edge and push what is needed next (Flannel), instead of heavy eager loads.	Cache scraped markdown keyed by URL + content hash; re-fetch only on change; prefetch the audit report the reviewer will most likely open next.
State sharded by consistent hash with seconds-level replacement of unhealthy hosts.	Shard Playwright/scraper workers by business ID hash; any worker is killable and replaceable so one hung audit never stalls the queue.
Tenant is the partition boundary; every row, event, and log line carries tenant identity.	Add client_id (agency customer) to every table, event, and log line now, before the first client signs; retrofitting tenancy is 10x harder.

Table 3. Slack-scale engineering principles and their IC application.

3.2 Security review of IC as found

The security posture today is "developer machine" grade, which is fine for a demo and unacceptable for client data. The FastAPI application exposes every endpoint without authentication: a grep for FastAPI dependency guards (Depends, api_key, Bearer, Authorization) in server.py returns zero matches. CORS is configured with allow_origins set to the wildcard, which tells browsers that any website may call the API. The production entry point runs uvicorn with reload=True, a development mode that watches files and restarts, and it exposes the interactive Swagger documentation at /docs by default. Data protection is equally thin: every lead, including names, phone numbers, and message history, is persisted as plain JSON files with no encryption at rest, and 89 of those files (9.8 MB of data plus 572 KB of logs) are committed into git history, which means even removing them going forward leaves them in past commits; the repository must be history-cleaned or the data rotated. On the positive side, no hardcoded secrets were found in the source tree, .env is correctly gitignored, and .env.example is complete and honest, which is a real and often-missed discipline.

The fix set is small and standard, and Chapter 9 schedules it in week 1 and 2: single shared-token auth on all dashboard and API routes (a FastAPI dependency checking a signed token from the environment), CORS locked to the dashboard origin, reload=True replaced by a real process manager, data/ and logs/ untracked with a corrected .gitignore plus git history filtering, and the ECC AgentShield scan (which your own vendored skill provides, and which exists precisely to catch hardcoded credentials, injection-prone calls, and undeclared dependencies) wired as a mandatory CI job. Outbound compliance should become a hard gate as well: the ECC outbox shield already defines grounded-claims-only, PII redaction, opt-out, and honest-sender rules, and those checks must run on every dispatch path, not live only in a module that nothing imports.

3.3 Verification as infrastructure

Your Maven course notes and the superpowers skill agree on the single most important rule: verification is not a phase, it is infrastructure. Concretely, IC needs three gates that run without a human remembering anything. Gate one is **pytest in CI** on every push, with the five broken imports fixed first, coverage reported, and merge blocked on red. Gate two is **AgentShield in CI**: the vendored ECC scanner runs against every diff and blocks hardcoded secrets, undeclared imports, and dangerous calls. Gate three is **evals for the agents**: a golden set of twenty real inbound conversations run through the Closer agent on every prompt change, scored for intent classification and booking conversion, so that prompt edits are measured the way harness2 proposes (propose, probe on a fixed set, compose the win into the harness). These three gates are what your Grok Bot masterclass notes called "CI rules that block failed merges," and they cost a weekend to stand up once the test suite is repaired.

> *Big-company software is not bigger software. It is the same software with machine-enforced truth: the tests, the shields, and the evals run themselves, so humans only argue about product.*

4. Reorganization: A Big-Company File Structure

You asked for a codebase and file structure "organized like big company software," with everything properly connected and anything not essential to IC separated out. This chapter gives that plan. The guiding rule is the one big companies actually follow: **the runtime is small, connected, and typed; everything else lives at the edges**. Skills, research notes, scratch scrapers, and vendored reference code never ship with the product; they feed the product through one interface (the skills directory for agents, the tool gateway for APIs). Today IC violates this rule in three ways: two generations of modules are both alive, three separate scraper trees exist, and 3,592 markdown skill files sit inside the runtime repository next to the server code. Nothing here requires deleting knowledge; it requires moving pieces to where a new engineer expects them.

4.1 The three problems to untangle

1. **Split-brain modules.** main.py and cli.py import the v1 tree (core.engine, services.llm_client, scrapers.scrapper_module, delivery.delivery_module) while the dashboard and pipelines import the v2 tree (src.*). Both work, neither is deprecated, and the tests mix them. One tree must win, and the natural winner is src/ because it has the contracts, models, and pipelines.

2. **Three scraper homes.** src/scrapers/ (six scrapers with a base class), root scrapers/scrapper_module.py (CSV, Google Places, OSM), and outreach/ (gmaps_scraper, sulekha_scraper, email_finder, reply_tracker, pain_point_analyzer). The outreach/ package in particular duplicates email finding and Google Maps logic that already exists in v2.

3. **Runtime and development kit merged.** .agents/skills (3,592 markdown files, 1,017 JS files), img/, ic-logo-concepts.svg, and show_leads.py sit beside the production server. They are developer tooling and brand assets, not runtime code, and they inflate every clone and every CI run.

4.2 The target structure

```
ic/
|-- apps/
|   |-- ic-api/                # FastAPI server, webhooks, dashboard routes
|   |-- ic-worker/             # queue workers: scrape, audit, copy, send
|   +-- ic-cli/                # typer CLI (cli.py, main.py merged)
|-- packages/
|   |-- ic-core/               # models, contracts, evidence_verifier, playboo
ks
|   |-- ic-scrapers/           # ONE scraper package: places, osm, gmaps,
|   |                          #   scrapling, firecrawl (deps declared)
|   +-- ic-delivery/           # twilio + smtp + telegram, idempotency keys
|-- dashboard/                 # UI split out of the 6,528-line html
|   |-- styles/tokens.css      # existing CSS variables, refined
|   |-- components/            # spotlight-card, console, drawer, hermes
|   +-- pages/                 # discover / auditor / outbox / deals
|-- skills/                    # the 51 vendored skills (dev-only, git-light)
|-- evals/                     # golden inbound threads, prompt evals
|-- infra/                     # Dockerfile, compose, CI workflows, envs
|-- docs/                      # architecture.md, ADRs, runbooks, GLOSSARY.md
+-- tests/                     # unit + integration + e2e (green, enforced)
```

Two decisions in this tree deserve explanation. First, the split into apps and packages is what makes the worker split of Chapter 3 possible: the queue workers import the same ic-core models and the same ic-delivery senders as the API, so there is exactly one implementation of "send a WhatsApp message" in the company, and it is idempotent. Second, dashboard/ is split only one level deep, not converted into a full React build system; your single-file HTML is actually an advantage for FastAPI deployment, and Chapter 6 upgrades it in place. If you later adopt the OpenUI streaming renderer for Hermes, the components directory gives that work a natural home without a framework migration.

4.3 Keep, connect, separate: what belongs to IC

Part	Verdict	Where it goes and why
src/ (models, contracts, pipelines, services, api, agents)	**Keep**	Becomes packages/ic-core, apps/ic-api, apps/ic-worker. This is the real product.
v1 root: core/, services/, scrapers/, delivery/, analytics/	Merge then delete	Port anything missing (analytics.track_event) into src/, repoint main.py and cli.py, delete the roots. One tree.
outreach/ package	Merge then delete	reply_tracker logic folds into inbound_closer; sulekha_scraper and duplicate gmaps/email finders fold into ic-scrapers or are dropped.
.agents/skills (51 skills)	**Separate**	Moves to skills/ at the monorepo root (or its own repo, vendored by path). Dev-only: agents read it; the server never imports it.

Part	Verdict	Where it goes and why
evidence_verifier, ecc_shield, contracts	**Keep + enforce**	Stay in ic-core and gain CI power: AgentShield scan in CI, ECC outbox gate on every dispatch path.
paperclip_bridge	Keep, optional	Stays as a service module behind a feature flag; when Paperclip is not running, the drawer shows a clean empty state (Chapter 6).
data/, logs/ tracked in git	Separate now	Untrack, gitignore, history-clean (D4). Runtime state belongs to volumes/backups, never version control.
img/, logo concepts, show_leads.py	Separate	Brand assets to docs/brand or a design repo; show_leads.py becomes a CLI subcommand or a test fixture.
harness_engine, dev_harness, deep_research, agency_swarm	Keep as lab	These are experiments (harness2-style). Keep them in packages but mark experimental; do not let them block the production path.
treg / firecrawl clients	**Connect**	Real integrations; declare deps, add tests, route through the internal gateway pattern (Chapter 5).

Table 4. Disposition of every major part of the current repository.

4.4 Migration checklist, in order

1. Untrack data/ and logs/ (git rm -r --cached), fix .gitignore, and run git history filtering so committed PII leaves history; rotate any credentials that ever touched the repo.

2. Restore src/storage/repository.py from the tests themselves: the five broken test files define the exact API (LeadRepository, SessionRepository, lead_repo). Make those tests pass without editing the tests.

3. Move analytics.track_event into src/, repoint main.py and cli.py imports from the v1 tree to src/, then delete the v1 root packages and outreach/ after merging unique logic.

4. Add LICENSE (your choice; MIT or Apache-2 are standard), fix the README file:/// link to a relative link, and reconcile the version (v2.5) across docs.

5. Create the apps/packages/dashboard/skills layout with git mv so file history survives; keep the CI-less repo green at every step.

6. Stand up .github/workflows/ci.yml: pytest, coverage report, AgentShield scan, and a docs-drift job that fails if folder_structure.md no longer matches the tree.

7. Declare scrapling and firecrawl in requirements (or extras) and add smoke tests for each scraper with recorded fixtures.

> *The test files are your specification. They already import the storage API you intended to build; building it to make them green is not new work, it is finishing work.*

5. The Unified Data Pipeline: From Scraping to CRM

You asked two pipeline questions: "did you add all scrapling tools and proper flow" and "how can the bot scrape a business, analyze its needs, build the right deliverable, and send it to their WhatsApp with a growth note." The answers: the scraping tools are integrated but degraded (Chapter 2, D7), and the end-to-end flow you described in your message is exactly the pipeline IC was built for, minus three links (inbox, CRM, and deliverable builder). This chapter defines the corrected, unified flow and the role of each researched tool in it. The figure below is the target architecture; every stage number in the text refers to it.

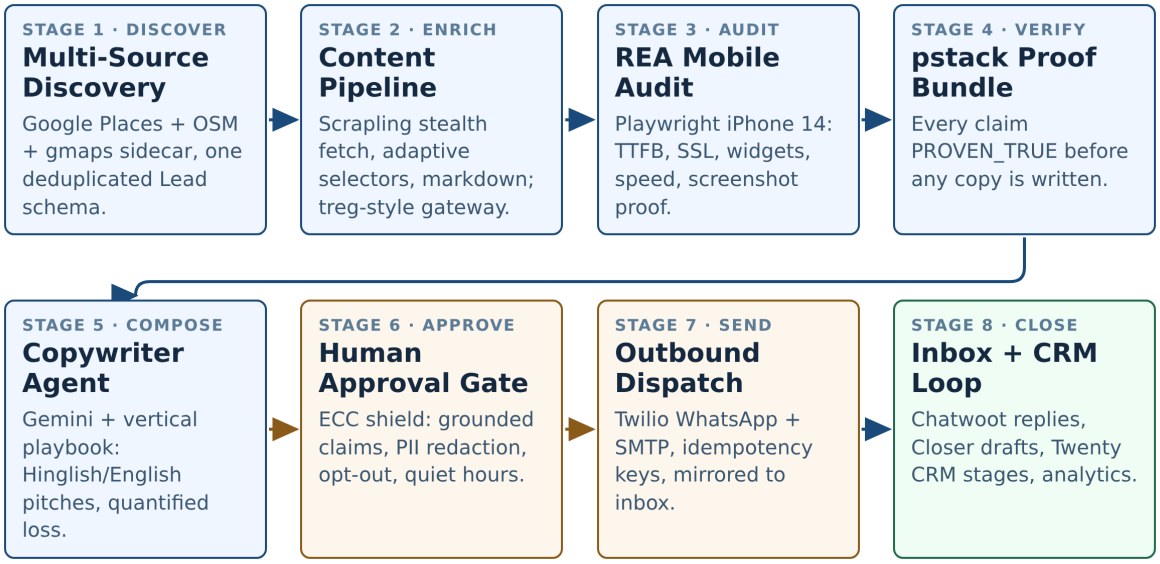


Figure 3. The unified IC pipeline: eight stages from discovery to the CRM loop (target state).

5.1 Discovery: three tiers, one schema

Keep the three-source design but assign it explicit tiers. **Tier 1 (primary): Google Places API** stays the default because it is ToS-compliant, structured, and stable. **Tier 2 (free): OSM Overpass** remains the zero-cost fallback for coverage gaps and testing. **Tier 3 (bulk): the gosom google-maps-scraper** runs as a Docker sidecar for whole-city grid crawls and email harvesting that Places Text Search does poorly; it drives a headless browser against Google Maps, which violates Google ToS, so it runs at low concurrency, with proxies, off-peak, and never as the default path. Every source normalizes into the same Lead schema with dedupe keyed on place_id or website. Your bookmark collection adds one more option here: ScrapeGraphAI (LLM-orchestrated scraping graphs), which is elegant for one-off odd sources but too slow and costly for bulk discovery; use it for special cases, not the firehose.

5.2 Enrichment and audit: Scrapling and Firecrawl, each doing its job

The research on both libraries is unambiguous about the division of labor. **Scrapling** (BSD-3, pure Python) becomes your default fetcher and parser: its Fetcher impersonates real browser TLS fingerprints, its StealthyFetcher handles Cloudflare-protected contact pages, its adaptive parser re-finds elements by similarity when a site redesigns (the number one maintenance cost in scraper fleets), and its .markdown() output gives the LLM clean content without paying Firecrawl credits. **Firecrawl** (AGPL core, cloud API) is reserved for what it alone does well: /v2/map to enumerate a prospect site, batch scraping with actions (click, scroll, wait), structured extraction into your AuditReport fields, and change monitors that re-audit a watchlist site when its content changes. Both must move from silent-fallback integration to declared dependencies with smoke tests. And critically: **the REA mobile audit stays yours**. Scrapling complements it; nothing replaces a controlled iPhone-14-viewport Playwright run with screenshots, because those screenshots are the proof artifacts your pitches are built on.

All paid or credentialed calls in this stage should pass through a **treg-style internal gateway**, whether or not you ever run treg itself: one FastAPI service holds every vendor key (SerpAPI, enrichment, proxies, Twilio), exposes tools with task-based search ("work email", not vendor names), publishes per-call price and success metadata, injects credentials server-side so agents never hold keys, and wraps every call in an idempotency key so a network retry can never double-bill. Register your own scrapers and audit endpoints as team tools and every teammate agent can call them without holding secrets. This single pattern eliminates the credential sprawl that AgentShield keeps warning about.

5.3 Outreach, inbox, and CRM: closing the loop

Outbound stays as built (Twilio WhatsApp, Gmail SMTP, Telegram alerts) with two additions from Chapter 3: idempotency keys and per-tenant rate limits. The big additions are the two systems your bookmarks already point at. **Chatwoot** (Rails + Postgres/pgvector + Redis + Sidekiq, one docker-compose) becomes the two-way inbox: provision one account per client via its Platform API, create an API-channel inbox per campaign, mirror every IC outbound message into it, and subscribe to its message_created and conversation_status_changed webhooks. Inbound replies then flow into IC in near real time, Gemini classifies intent (interested, pricing objection, agency objection, unsubscribe), the ECC outbox gate checks the draft, and the reply is sent back through Chatwoot so threading and assignment stay intact. This converts fire-and-forget broadcasting into an actual sales conversation, which is what your Closer agent was designed for. **Twenty** (NestJS + GraphQL + Postgres, React front) becomes the CRM: IC writes Company and Person records at discovery, and moves Opportunity stages as the funnel advances (New, Audited, Contacted, Replied, Negotiating, Won). Deploy Twenty as your CRM backend rather than building CRM tables yourself; its data model already anticipates SDR work with timeline, notes, calls, and calendar objects.

From **rakazo** (the persistent AI-teammates platform by the Inbox Zero author) steal patterns, not the platform: durable per-agent routines (your daily scrape-audit-draft cadence, owned by an identity with memory), a per-campaign working journal that agents append structured entries to (the Slack context-engineering lesson: role-tailored summaries plus citations, never raw transcripts), human takeover of an agent session, delegation to short-lived subagents, and a hash-pinned skills registry so every agent runs the same auditable recipes. Your vendored harness_engine and scheduled_cadence services already gesture at these; the journal and takeover patterns are what make the Closer agent feel like a teammate instead of a chat window.

5.4 The WhatsApp deliverable bot, formalized

Your message described the outreach you want: the bot scrapes a business, analyzes what it needs, builds the right deliverable (website, app, or automation), and sends it to the business WhatsApp with a short note about how their website can grow the business, inviting them to make contact. That is IC Stages 1 through 7 with one new service in the middle. The formalized flow: (1) discover and enrich the business through the pipeline above; (2) the REA audit plus review-mining produce a quantified problem list (load seconds, missing SSL, no WhatsApp ordering, no booking); (3) a **deliverable router** decides which of your three products fits the dominant pain: no or broken mobile site suggests a website build, manual booking or quoting suggests an automation, fragmented customer handling suggests a mini-app or CRM setup; (4) IC generates the deliverable or a concrete preview (a one-page proposal with the audit screenshots, and for website leads optionally a rendered mock homepage); (5) the CopywriterAgent writes the short Hinglish/English growth note, grounded only in verified claims; (6) the ApprovalGate human reviews and approves; (7) Twilio sends it to the business WhatsApp; (8) Chatwoot captures the reply and the Closer takes over. Guardrails are non-negotiable: every number in the note traces to a proof bundle, opt-out language in every first message, WhatsApp Business template compliance, quiet hours per the target region, and a per-tenant send ceiling so a bad batch can never spam a market.

Tool	Role in IC	Effort	Priority
Scrapling	Default fetch/parse, stealth contact-page fetch, adaptive selectors, markdown for LLM	S	Week 2
Firecrawl (cloud)	Site map, batch scrape, structured extraction, change monitors	S	Week 2
gosom gmaps-scraper	Bulk/grid discovery sidecar, email harvesting (ToS-cautioned)	M	Week 3+
treg (pattern)	Internal tool gateway: server-side secrets, idempotency, price metadata	M	Week 2-3
Chatwoot	Unified two-way inbox, webhooks to Closer, per-client accounts	M-L	Week 3
Twenty	CRM data model + record table/kanban UI patterns	M	Week 3
rakazo (patterns)	Routines, agent journal, human takeover, skills registry	S-M	Week 4
ScrapeGraphAI	Occasional LLM-graph scrapes for odd sources (not bulk)	S	Optional

Table 5. Tool adoption summary (from the researched repositories).

6. UI/UX Blueprint: From Dashboard to Agency OS

This chapter turns nineteen researched sources into a concrete, anti-slop UI plan for IC. The current dashboard is closer to big-company quality than you may realize: it already has a dual-theme CSS custom-property system (--bg-app, --bg-card, --bg-sidebar, --font-sans, --font-mono), seventeen keyframe animations, react-bits-style SpotlightCard patterns, and a four-step workflow (Discover, REA Auditor, Outbox, Hermes Deals Desk) plus the Paperclip drawer. The problem is not taste; it is structure (one 6,528-line file), missing power-user patterns (command palette, record side panel), and unmanaged indulgence risk (animation and glass with no budget). The plan below is sequenced so that structure comes first, patterns second, character third.

6.1 Structure first: the token and layout foundation

Keep the FastAPI-served, vanilla-JS architecture; it is a genuine advantage (no build step, instant deploys) and every pattern below works in plain HTML. Split the file into styles/tokens.css, components/, and pages/ per Chapter 4. Formalize the token system with a scale you can defend: one accent (the existing blue family), one semantic set (success, warning, danger, info), spacing in a 4-point grid, radii at 6/10/14 px, and exactly two font families (the existing sans + mono pairing, which already reads like a console product). Typography discipline: 15-16 px body, 12-13 px secondary, 700-weight only for page titles and KPI numerals, and never letterspaced all-caps on small labels (the Hallmark sloplint rule your research flagged). Light and dark stay theme-switchable through the same variables, exactly as Twenty does with its generated --t-* variables and .dark class.

6.2 The twelve upgrades, ranked

#	Upgrade	Source	Where in IC	Effort
1	SpotlightCard edge-light + Magic Bento grid	react-bits	Discovery metrics and KPI tiles	Low
2	Counter odometer + Status Mark dots	react-bits micro	Leads found, sends, queue depth	Low
3	Command palette (Ctrl-K) + inline AI bar	Watermelon UI / Twenty	Global navigation and Hermes input	Low-Med
4	Benefit-verb CTA copy rewrite	cta.gallery	Outbox approve/send, all forms	Copy only
5	checklist.design web-app QA gates	checklist.design	Definition-of-done per screen	Process

#	Upgrade	Source	Where in IC	Effort
6	Glass Surface reserved for the iPhone bezel	react-bits / glass3d	REA auditor device toolbar only	Medium
7	One hero background (Aurora/Silk) with reduced-motion fallback	react-bits	Landing/empty Paperclip drawer header	Medium
8	Decrypted/Scrambled + Type text for agent states	react-bits text	Hermes "reasoning", score reveals	Low
9	glass3d recipe (noise + inset bevel) on drawer cards	glass3d.dev	Paperclip drawer, max 1 surface	Low
10	useSound: soft click + one chime, muted default	soundcn (CC0 only)	Approve/reject, new lead batch	Low
11	OpenUI Lang streaming renderer for Hermes	thesysdev/openui	Deals desk streams tables/charts	Med-High
12	sloplint-style pre-ship self-audit	usehallmark	Every release	Process

Table 6. Ranked UI upgrades with sources and difficulty (from the researched libraries and galleries).

6.3 Screen-by-screen, the big-company patterns

Step 1, Discovery. Adopt Twenty's record table: sticky header, inline cell editing, column visibility and reorder, and a right-side record panel that opens the lead's timeline (scrape runs, audit scores, messages) instead of a page navigation. KPI tiles use SpotlightCard with the odometer Counter. Empty state: a 3dicons CC0 map-pin render, one sentence of what will appear, one ghost CTA ("Run first discovery"), never a second filled button. **Step 2, REA Auditor.** This is your signature screen and it deserves the single glass surface in the app: the iPhone-14 bezel toolbar uses the glass3d recipe (blur capped at 20 px, noise overlay, inset highlight), the snapshot loads with a skeleton shimmer, and the audit findings list uses Status Mark dots (pass, warn, fail) with the proof reference one click away. **Step 3, Outbox.** Side-by-side WhatsApp/email preview stays; CTAs are rewritten per cta.gallery rules: "Approve and send 12 messages" (benefit + scope), risk-removal microcopy under irreversible actions ("Messages send immediately; review before approving"), and one filled button per view. Approval celebrates: a subtle check animation and the predicted reply-rate delta (the builtformars aha-moment rule). **Step 4, Hermes Deals Desk.** Keep objection pills; add streaming responses that preserve scroll position and copy-on-hover (the OpenIntelligentUI polish), starter suggestion chips that run real agent calls, and honestly-labeled sample data. Then, as the strategic upgrade, adopt OpenUI Lang so the agent can stream a sortable deals table or pipeline chart instead of markdown text. **Paperclip drawer.** Agent org chart as quiet cards with presence dots (in-memory only, the Slack presence lesson); when the bridge is down, the glass3d empty card says exactly what is offline and how to start it.

Navigation and chrome. From navbar.gallery: a slim topbar with breadcrumbs for the four steps, a persistent left sidebar with favorites (pinned campaigns and prospects, Twenty's pattern), a Ctrl-K command palette for jumping to any lead or action, and an announcement banner slot for system notices (rate limits, provider status). From footer.design's "small type" pattern: one quiet footer row with environment, version, API latency, and support link. Iconography: one system (lucide-style line icons) for all chrome; 3dicons CC0 renders appear only in empty states and success moments, never mixed into toolbars. Sound: soundcn modules with inline base64 (keeps the single-asset deploy), muted by default, two sounds maximum, and never the copyrighted WoW pack.

6.4 Anti-slop guardrails (the rules that keep it clean)

1. One WebGL background maximum, never behind data-dense surfaces; pause when the tab hides; respect prefers-reduced-motion; provide a CSS-gradient fallback.
2. Glass is seasoning, not material: one glass surface per view (the iPhone bezel), blur capped near 20 px, always with the noise + inset-edge recipe or it reads flat-frosted.
3. Exactly one filled CTA per view; everything else is ghost or outline (the sloplint "two filled buttons" rule).
4. No cursor toys, trails, or splash effects in work surfaces the team uses for hours; those live on a marketing landing page only.
5. Contrast discipline: no gradient or glow text on small labels; no text over imagery without a scrim; KPI numerals stay solid.
6. Motion budget: transitions 150-300 ms ease-out; decorative loops at least 10 s and subtle; one "wow" moment per screen maximum.
7. Faint texture only: dot grids or squares at 3-6 percent opacity; no CRT/terminal backgrounds in a professional console.
8. One icon system for chrome; 3D renders only in empty states and celebrations.
9. Sound off by default; two sounds per flow maximum; CC0 licenses only.
10. Copy humility: honest urgency only, error text always cause plus next step (microcopy.me pattern), and sample data labeled "illustrative, not live."

Finally, the QA discipline that makes this durable: before each release, run the checklist.design web-app checklist for every shipped screen (Dashboard, Empty State, Data Table, Notifications, API Keys, Audit Log, Search Results), plus the sloplint pass (contrast on imagery, double-filled buttons, gradient misuse, tracked-caps labels). These two process gates cost nothing, and they are what will make the UI feel consistently engineered rather than occasionally inspired.

7. Paperclip, REA, and pstack: What They Are and How Your Team Uses Them

You asked directly: what is Paperclip useful for in the UI, what is the purpose of the REA inspector and the iPhone 14, why did we add pstack and the REA inspector, did I add these things, and how should the team actually use the skill stack day to day. This chapter answers each question in plain language, and then gives the weekly team flow table that maps every skill to a specific moment in your development week. First, the direct answer about provenance: **yes, all of these are already in your repository**. The 51 skills including pstack, superpowers, mattpocock-skills, ecc, treg, and harness2 are vendored under .agents/skills/; the Paperclip bridge is implemented in src/services/paperclip_bridge.py; the REA-style auditor with the iPhone 14 viewport is implemented in the site inspector and visual auditor services and surfaced in dashboard Step 2; and the pstack evidence engine is implemented in src/services/evidence_verifier.py. What is missing is not the code; it is enforcement (CI gates) and habit (the weekly flow below).

7.1 Paperclip: the workforce UI for your agents

Paperclip is a multi-agent workforce manager: a separate application (running on localhost:3100 in your setup) where agents appear as an org chart of workers with roles, task queues, and live status. Its purpose in IC is to be the **management UI for the agent team**, the way a real agency has a bench of specialists. Concretely, the bridge module (paperclip_bridge.py) connects IC's FastAPI app on port 8000, the Paperclip server on port 3100, and a React studio on port 5173, and can fetch the active company record and the list of registered agents; the dashboard's "Paperclip Live Studio" drawer embeds that view so you can watch and steer the workforce without leaving IC. How to use it well: register each IC agent (Scout, Auditor, Copywriter, Closer) as a Paperclip worker so assignment and status are visible in one place; use the drawer for daily standup-style checks (who is busy, what is queued, what failed); and treat it as optional infrastructure, which is why Chapter 4 puts the bridge behind a feature flag with a clean empty state when Paperclip is not running. The value is operational visibility and a mental model your whole team shares; the risk to avoid is making the product depend on a second server to boot.

7.2 REA and the iPhone 14 inspector: the proof machine

REA (in this codebase's vocabulary, "Reverse-Engineering Anything," in the style of Chrome DevTools) is the discipline of inspecting a live product to extract the truth about how it behaves. The REA inspector in IC is the AuditorAgent: it opens a prospect's website in a headless Playwright browser emulating an **iPhone 14 viewport of 390 by 844 pixels**, and measures what a real mobile visitor experiences: time-to-first-byte, SSL validity, mobile responsiveness, whether a WhatsApp ordering widget (wa.me) exists, whether one-click booking exists, and it captures a genuine mobile screenshot as evidence. Why the iPhone 14 specifically: it is the reference device for modern mobile-audience testing (a mainstream, current-generation viewport), the business owners you target are overwhelmingly visited on phones, and a fixed canonical viewport makes audits comparable across thousands of businesses and over time. Why it matters to the business model: the audit is what turns "your website could be better" into "your site takes 6.1 seconds to load on a phone, has no WhatsApp button, and no booking; that is where your after-hours leads are going," and the screenshot is the artifact the prospect cannot argue with. In the UI, Step 2 shows the phone bezel with the live snapshot; that visual is your credibility. Use it by running audits in batches, letting the evidence verifier certify each finding, and never writing a pitch line the audit cannot prove.

7.3 pstack: the rigor stack and the proof bundles

pstack is Lauren Tan's (Poteto) engineering skill stack for agents, the system she describes as turning an AI coding tool into "a real engineering team": playbooks for tasks, adversarial review, and above all the rule that claims require proof. In IC it plays two roles. In **development**, the vendored pstack skills (interrogate, blast-radius, visual-parity, hillclimb, show-me-your-work) are how your team reviews diffs and tunes metrics; visual-parity is tailor-made for the dashboard work in Chapter 6 (pixel-comparing a rebuilt screen against the old one). In **runtime**, evidence_verifier.py is pstack's "prove it works" principle embodied: every audit finding becomes a VerifiedClaim with an evidence_type (live HTTP probe, Playwright screenshot, web citation, DOM selector), a proof_reference, and a PROVEN_TRUE status, assembled into a PstackAuditBundle with a proof score. That bundle is why the CopywriterAgent cannot hallucinate a statistic into a pitch and why the ECC outbox gate has something to check against. Did you add it? Yes: the skills are vendored and the verifier is wired into the lead model; the remaining work is making the bundle mandatory in the outreach pipeline and showing the proof score in the Outbox UI.

7.4 ECC: the shield on both sides of the wall

ECC (Everything Claude Code, affaan-m) is a dual-engine harness, and IC uses both engines. The **development engine** (AgentShield) scans code for hardcoded credentials, injection-prone calls, dangerous eval/system usage, and imports that are missing from requirements; this is precisely the scan that would have caught your two declared-but-undeclared scraper dependencies, and it belongs in CI starting week 1. The **runtime engine** is the one your product needs more than most: before any outbound message, it enforces grounded claims only (every number traceable to the AuditReport), no hallucinated names or statistics, PII and internal-data redaction, and spam/phishing hygiene (accurate sender, opt-out, respectful tone); after an inbound reply, its Hermes reviewer checks the Closer's draft before it is sent. Your repo already encodes these rules in src/agents/ecc_shield.py with tests; the gap is that nothing forces every dispatch path through the gate. Wire it at the single send function in ic-delivery and the gate becomes load-bearing.

7.5 The weekly team flow with the full skill stack

When	Activity	Skill(s) used
Mon AM	Pick the week's work; get grilled until the design resolves; update glossary and docs	mattpocock grill-with-docs, brainstorming
Mon PM	Write the spec, break into tickets; pick UI direction with 2-3 variant prototypes	mattpocock to-spec, to-tickets; pstack arena / prototype
Tue-Wed	Branch per feature in isolation; plan 2-5-minute tasks; implement test-first	superpowers using-git-worktrees, writing-plans, subagent-driven-development, test-driven-development
Tue-Wed UI	Build screens to pixel parity with the mock; verify iPhone-14 renders of the dashboard	pstack visual-parity; ECC design-token review
Any day	Root-cause bugs with discipline, not vibes	superpowers systematic-debugging; mattpocock diagnosing-bugs
Thu AM	Review every PR on two axes (standards, spec) plus an adversarial pass	superpowers requesting/receiving-code-review; mattpocock code-review; pstack interrogate
Thu PM	Harden for release: e2e the approve-to-send path, security scan, check what a change breaks	ECC e2e + security-scan (AgentShield); pstack blast-radius
Thu late	Release gate: fresh evidence or the claim is a lie	superpowers verification-before-completion
Fri AM	Merge, clean worktrees, tag the release	superpowers finishing-a-development-branch

When	Activity	Skill(s) used
Fri PM	Retro, save session memory, update Closer prompts from the week's outcomes, scout leads for next week	mattpocock retro; ECC learn/save-session; harness2 propose-probe-compose; treg lead-signals

Table 7. How the team uses the vendored skills across a week (2-5 engineers, building IC and shipping client work).

Two skills in your vendored stack deserve a note beyond this table. **harness2** (Google Research) is a methodology donor, not a daily driver: once a month, run a propose-probe-compose loop on the Closer agent, propose prompt or tool changes, probe them on the golden set of real inbound threads in evals/, keep only measurable wins, and record the lesson in a LESSONS.md that future edits must consult. **treg** is both a skill and a product pattern; the skill teaches agents to search a tool catalog by task and disclose per-call cost before spending, and even without the hosted service the pattern (server-side secrets, idempotency keys, price metadata) becomes your internal gateway as described in Chapter 5. That is the full stack, and none of it requires anything you do not already have on disk.

8. Market Expansion: High-Pain Categories Beyond the Three Verticals

You asked to look past the current three target categories (Freight and Transport, Events and Venues, Local SMBs and Clinics) at the categories with the most market pain, where businesses want to automate, what problems they face, and what IC should build for them. The selection rule comes from how IC works: the pain must be **detectable from the outside** (the REA audit and Google Maps presence must reveal it), quantifiable (so the pitch carries a number), and solvable with something a small agency can deliver (website, automation, or CRM setup). The cross-cutting statistics below anchor every vertical, and each has sources listed in the appendix.

78%
of customers buy from the vendor that responds first

7×
more likely to qualify a lead when responding within one hour

53%
of mobile visits abandon pages loading over 3 seconds

62%
of calls to small businesses go unanswered

Two of those numbers deserve sourcing care: the 78 percent first-responder figure and the 62 percent missed-call figure are widely cited industry statistics rather than peer-reviewed studies; the 7× and 42-hour figures come from the Harvard Business Review study of online lead response ("The Short Life of Online Sales Leads"), and the 53 percent and 3-second benchmark is Google/SOASTA mobile research, with Deloitte's "Milliseconds Make Millions" adding that a 0.1-second speed gain correlates with roughly 8 percent more retail conversions. Also relevant: about 27 percent of small businesses still have no website at all, which is itself an offer (a fast one-page site with WhatsApp and booking). Every

vertical below fits your existing audit-plus-outreach pipeline without new engineering, because the detecting signals are exactly what AuditorAgent already measures.

Vertical	Core pain (quantified)	Deliverable to sell	Detectable audit signal
Real estate brokers	Speed-to-lead decides sales; portal leads decay in minutes; brokers respond after hours or never	Instant WhatsApp/SMS response bot, listing site with booking link, lead CRM	No chat or booking widget, static form, slow mobile site
Law firms (PI, family, immigration)	Intake latency loses retainers; industry-average response is measured in days (HBR 42-hour average)	24/7 intake bot, after-hours WhatsApp triage, consultation booking, SSL and speed fixes	No live chat, no booking, HTTP-only, 5 s+ mobile load
Home services (HVAC, plumbing, roofing)	Missed calls are lost high-ticket jobs; emergency intent expires in minutes; 62/85 percent missed-call figures apply brutally	Missed-call-to-text automation, quote follow-up, review engine, fast mobile site	No SSL or no site, no booking, incomplete Maps profile
Gyms and fitness studios	Trial inquiries go cold without sub-hour follow-up; manual class booking friction	Booking widget, WhatsApp trial follow-up and win-back sequences, landing page	PDF schedules, no online booking, Instagram-only presence
Salons and spas	20-30 percent no-shows; aggregator commissions on every booking	Direct WhatsApp confirmations and reminders, own booking page, rebooking automation	Booking only via third-party link, no reminder channel
Restaurants	15-30 percent aggregator commissions on direct-able orders; reservations lost to voicemail	Direct-order and reservation flow, WhatsApp order updates, speed fixes	Menu-PDF site, only aggregator deep-links, slow LCP, no schema markup
Education and coaching institutes	Parent inquiries pile up unanswered; one lost cohort costs thousands per student	Enquiry-qualification bot, demo-class booking, follow-up CRM cadence	Enquiry form with no automation, no WhatsApp CTA, no booking
Car dealers and repair	Test-drive and quote leads answered in hours instead of minutes; manual service reminders	Lead-response bot, service booking widget, inventory site with SSL	Outdated site, no chat, no booking, missing SSL
Wedding vendors	Couples message 5-10 vendors and book the first substantive reply; seasonal floods overwhelm	WhatsApp qualifier bot, portfolio site, calendar booking, price-guide automation	Instagram-only presence, no booking link, weak mobile experience

Table 8. Nine additional high-pain verticals, quantified pain, the deliverable to sell, and the audit signal that detects it.

Adding a vertical to IC is deliberately a data change, not a code change, because of the architecture you already have. The recipe: extend vertical_playbooks.py with a detection profile for the category (the signals in the last column of Table 8), add the pitch claims to pitch_claims.yaml so the CopywriterAgent can only quote pre-approved numbers for that vertical, and let detect_vertical() route new leads automatically. The deliverable router from Chapter 5.4 then decides whether the right offer is a website, an automation, or a CRM setup, and the WhatsApp growth note follows the same approval gate as every other vertical. Sequence the expansion by ease of detection and ticket size: home services and real estate first (strongest detectable signals, high ticket), salons and restaurants next (aggregator bypass is an easy quantified pitch), then education, wedding vendors, and car dealers.

9. Four-Week Engineering Plan and KPIs

Everything above collapses into four weeks of focused work for a two-to-five person team. The sequencing principle is truth first, then doors, then flow, then face: repair verification and the repository (week 1), lock security and the merge (week 2), upgrade the pipeline and close the loop (week 3), and finish with the UI sprint and the eval harness (week 4). Each week ends with a shippable state, and the skill stack of Chapter 7 governs the work itself. Do not skip ahead: every later week depends on the CI gate being green.

9.1 The plan

Week	Theme	Deliverables
1	Truth (P0 fixes)	Restore src/storage/repository.py until the 5 broken test files pass unmodified; fix the 1 failing + 8 errored tests; untrack data/ and logs/ and history-clean; add LICENSE; fix README paths and version drift; stand up CI (pytest + AgentShield + docs-drift check).
2	Doors and one tree	Shared-token auth on all routes; CORS locked; reload=True replaced; single module tree (v1 and outreach/ merged, deleted); scrapers declared in requirements with smoke tests; idempotency keys on every outbound send; per-provider rate limiters.
3	Flow	Job queue (Redis + arq) for scrape/audit/send workers; Chatwoot deployed with per-client accounts, campaign inboxes, and webhooks into the Closer; treg-pattern internal gateway holding vendor keys; Twenty deployed as CRM; deliverable router (website / automation / mini-app) behind the copywriter.
4	Face	Dashboard split into components/pages; the twelve UI upgrades of Table 6 in ranked order; checklist.design and sloplint gates added to the release process; evals/ seeded with 20 golden inbound threads and the monthly harness2 loop scheduled; optional: Paperclip agent registration for workforce visibility.

Table 9. Four-week engineering plan (P0 = trust-critical).

9.2 KPIs that prove the upgrade

KPI	Today (measured)	Week 4 target
Test suite	33 pass / 1 fail / 8 errors / 5 uncollectable	All green in CI on every push
CI gates	None	pytest + AgentShield + docs-drift, blocking
Dashboard auth	None, CORS wildcard	Token auth, CORS locked, /docs gated
PII in git history	89 tracked files (9.8 MB)	Zero; runtime state in volumes
Scraper coverage	3 trees, 0 tested, deps undeclared	1 package, smoke-tested, deps declared
Outbound safety	Manual gate only	Idempotent sends, rate limits, ECC gate on dispatch
Reply loop	One-way send	Chatwoot inbox, Closer drafts, approval, CRM stages
UI structure	1 file, 6,528 lines	Tokens + components + pages, 12 upgrades live

Table 10. Target KPIs before and after the four-week plan.

The first five actions to take today, in order: run pytest yourself and read the five import failures (they are the specification for the storage layer); git rm -r --cached data logs and fix .gitignore; add the LICENSE file; write the twelve-line CI workflow; and register your first Chatwoot inbox connected to a test WhatsApp number. Everything else in this report is built on those five moves.

> *The system you set out to build, a trustworthy agent that earns autonomy through proof, is the system IC's architecture already describes. The remaining work is to make the machinery enforce what the documentation promises, and that is a month of disciplined work, not a rebuild.*